

Day 10 of training

-->Switch Statement in JavaScript

The switch statement in JavaScript is used to perform different actions based on different conditions.

It provides an alternative to using multiple if-else statements, making the code more readable and organized.

```
switch(expression) {  
  
  case value1:  
  
    // Code to execute if expression === value1  
  
    break;  
  
  case value2:  
  
    // Code to execute if expression === value2  
  
    break;  
  
  ...  
  
  default:  
  
    // Code to execute if no case matches  
  
}
```

How It Works

The expression is evaluated once.

The value of the expression is compared with the values of each case.

If a match is found, the corresponding code block is executed.

The break statement prevents the execution from falling through to the next case.

The default block is optional and runs if no matching case is found.

-->Ternary Operator in JavaScript

The ternary operator is a concise way to write conditional statements.

It is also known as the conditional operator and is used as a shorthand for the if-else statement.

Syntax:

```
condition ? expressionIfTrue : expressionIfFalse;
```

-->Alert in JavaScript

The alert() method in JavaScript is used to display a popup message box to the user.

It is commonly used for notifications, warnings, or debugging purposes.

Syntax:

```
alert("Your message here");
```

-->Confirm Box in JavaScript

The confirm() method is used to display a popup dialog box that asks the user to confirm or cancel an action.

It returns a boolean value based on the user's choice.

Syntax:

```
confirm("Are you sure?");
```

How It Works

Shows a dialog with a message, OK, and Cancel buttons.

If the user clicks OK, it returns true.

If the user clicks Cancel, it returns false.

-->Prompt Box in JavaScript

The prompt() method is used to ask the user for input through a popup dialog box.

It displays a text box where the user can type and either press OK or Cancel.

Syntax:

```
prompt("Enter your name:");
```

How It Works

If the user clicks OK, it returns the input as a string.

If the user clicks Cancel, it returns null.

--> Functions in JavaScript

A function in JavaScript is a reusable block of code designed to perform a particular task.

Functions help organize code, reduce repetition, and improve readability.

Syntax:

```
function functionName(parameters) {  
    // code to execute  
}
```

Example:

```
function greet(name) {  
    console.log("Hello, " + name + "!");  
}
```

```
greet("Opinder");
```

-->Return Statement

A function can return a value using the return keyword:

Advantages

Reusability

Better code organization

Easy debugging and maintenance

-->. important for interview point of view

--->Global and Local Scope in JavaScript

In JavaScript, scope refers to the visibility or accessibility of variables.

Variables can be globally scoped or locally scoped, depending on where they are declared.

->Global Scope

A variable declared outside any function or block is in the global scope.

It can be accessed anywhere in the program.

```
let globalVar = "I am global";  
  
function showGlobal() {  
    console.log(globalVar); // Accessible here  
}
```

->Local Scope

A variable declared inside a function or block is in the local scope.

It can only be accessed within that function or block.

```
function showLocal() {  
    let localVar = "I am local";  
    console.log(localVar);  
}  
  
// console.log(localVar); // Error: localVar is not defined
```

->Block Scope with let and const

let and const are block-scoped.

They are only accessible inside the block {} where they are defined.

```
{
  let a = 10;
  const b = 20;
  console.log(a, b); // Accessible here
}
// console.log(a, b); // Error: a and b are not defined
```

->Function Scope with var

var is function-scoped, not block-scoped.

If declared in a block but outside a function, it becomes global.

```
function example() {
  if (true) {
    var x = 5;
  }
  console.log(x); // Works because var ignores block scope
}
```

| Keyword | Scope Type | Hoisted | Block Scoped | Reassignable |

Keyword	Scope Type	Hoisted	Block Scoped	Reassignable
`var`	Function scope	Yes	No	Yes
`let`	Block scope	No	Yes	Yes
`const`	Block scope	No	Yes	No